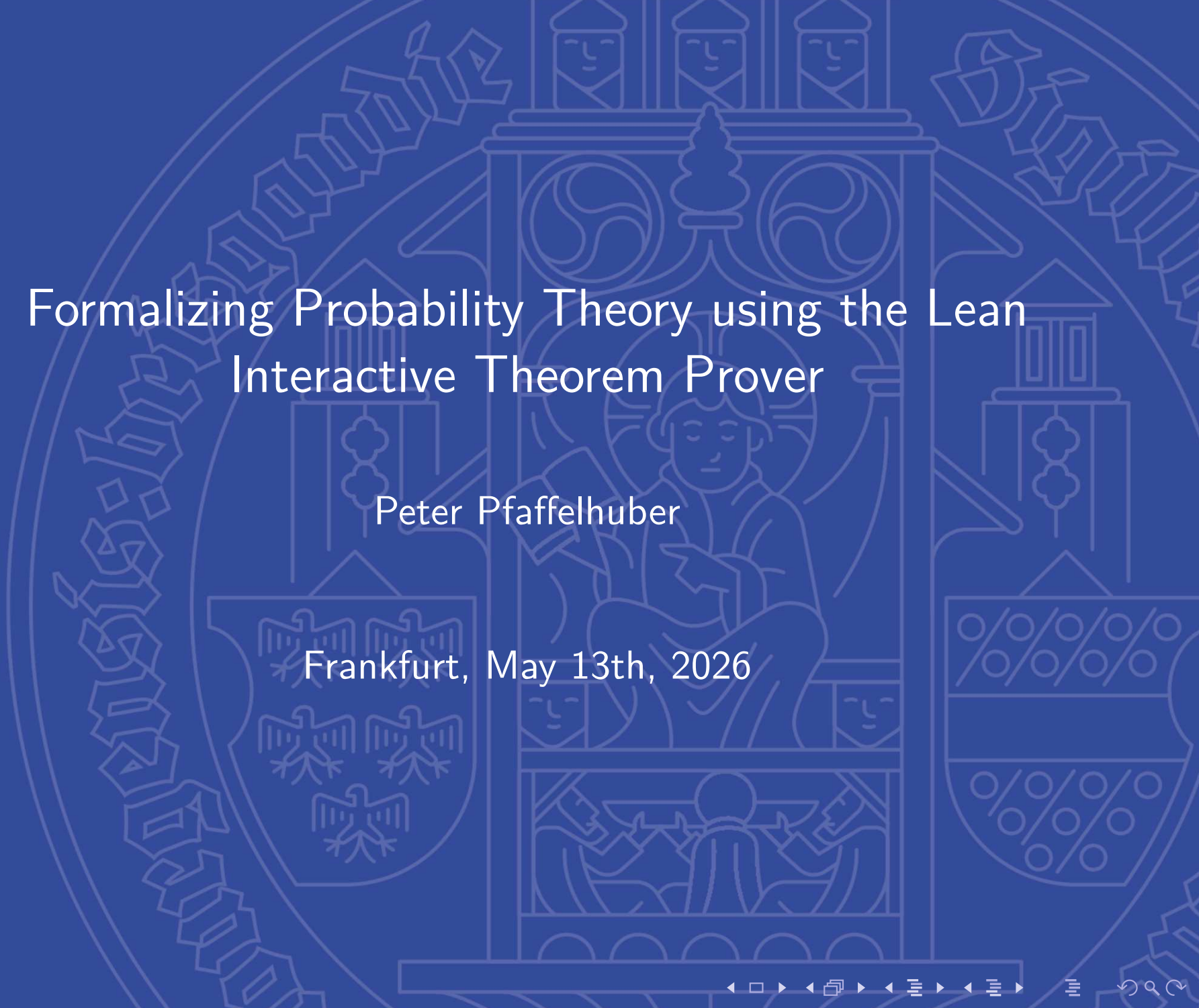


Formalizing Probability Theory using the Lean Interactive Theorem Prover

Peter Pfaffelhuber

Frankfurt, May 13th, 2026

Some LEAN pioneers

Leonardo di Moura

started Lean in 2013 at Microsoft.

Kevin Buzzard

promotes Lean for mathematicians since 2018.

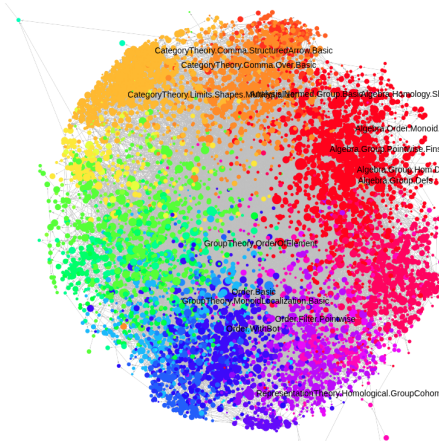
Johan Commelin

led the Liquid Tensor Experiment.

Terrence Tao

leads collaborative formalization efforts.

- Lean is a functional programming language.
- Developed by the Lean FRO (≈ 20 people).
- Key features: meta-programming, interactive theorem proving.
- The largest mathematical library is mathlib, with about $2 \cdot 10^6$ lines of code.
- The graphic shows the import structure of mathlib.



LEAN demo

- Divisibility
- Unintuitive behavior

- Every *term* has a *type*, e.g.:
  n : ℕ or α : **Type** or
  negneg_of_classical (p : **Prop**) : (p ∨ ¬ p) → (p ↔ ¬¬ p)
- There is a hierarchy of types: **Type** 0 has type **Type** 1 (in order to avoid logical pitfalls).
- Every type can be constructed, e.g.:
  inductive ℕ where
  | zero : ℕ
  | succ (n : ℕ) : ℕ
  The construction of a **Prop**-type is its proof.

- Lean uses *dependent types*. Example:
  def Vec (α : **Type**) (n : ℕ) := { v : List α // v.length = n }

  Type Vec, which depends on the term $n : \mathbb{N}$

  Errors in the length of a Vec are discovered at compile-time, not at run-time.

- Successful compilation of a lean file $\iff$ all proofs in that file are correct.

- The Curry-Howard isomorphism is described as *proofs-as-programs* and *propositions-as-types*. Example:
  ∀ x : **Type***, x = x := **fun** x => rfl

Discrete Probability

```
structure DiscreteMeasure (α : Type*) where
  weight : α → ℝ≥0∞
```

```
noncomputable def toMeasure [MeasurableSpace α] :
  DiscreteMeasure α → Measure α :=
  fun μ ↦ Measure.sum (fun a ↦ μ.weight a • Measure.dirac a)
```

```
noncomputable def coin (p : unitInterval) :
    DiscreteMeasure Bool where
  weight
  | true  => ↑p
  | false => ↑(1 - p)
```

Discrete Probability

```
noncomputable def binom (p : unitInterval) :
    ℕ → DiscreteMeasure ℕ
  | 0 => pure 0
  | n + 1 => do
    let X ← coin p
    let Y ← binom p n
    return (↑X + Y)
```

```
theorem binom_eq_count_true (p : unitInterval) (n : ℕ) :
    binom p n = do
      let X ← iidSequence n (coin p)
      return X.count true
```

The probability monad

- pure a = Dirac measure δ_a.
- μ.map f = pushforward: law of $f(X)$ when $X \sim$.
- Ξ.join = mixture: average over a random measure $\mu \sim \Xi$.
- μ.bind f = sample $X \sim$, then $Y \sim f(X)$.

```
noncomputable def map (g : α → β) (μ : DiscreteMeasure α) :
    DiscreteMeasure β :=
  ⟨fun b ↦ ∑' a, (g⁻¹' {b}).indicator μ.weight a⟩

noncomputable def join (Ξ : DiscreteMeasure (DiscreteMeasure α)) :
    DiscreteMeasure α :=
  ⟨fun a ↦ ∑' μ, Ξ μ * μ a⟩

noncomputable def bind (μ : DiscreteMeasure α)
    (f : α → DiscreteMeasure β) : DiscreteMeasure β :=
  (μ.map f).join
```

The probability monad

These operations satisfy the monad laws, the most involved being associativity of bind:

```
theorem bind_bind (μ : DiscreteMeasure α)
    (f : α → DiscreteMeasure β) (g : β → DiscreteMeasure γ) :
    (μ.bind f).bind g = μ.bind (fun a ↦ (f a).bind g)
```

```
instance : LawfulMonad DiscreteMeasure
```

This unlocks **do**-notation, which desugars to nested binds:

```
do
  let X ← μ
  let Y ← ν
  return (f X Y)
-- is the same as
μ.bind (fun X ↦ ν.bind (fun Y ↦ pure (f X Y)))
```

Discrete Probability (Insights from formalization)

- Discrete probability theory is like probability theory without any measurability issues.
  (This makes discrete probability measures a lawful monad.)
- Using **do**-Notation requires use of monads, and make definitions more readable.
- The binomial distribution can be defined in several equivalent ways.
- Formalizing basic mathematics already feels like developing a theory.
- Mathematically easy things can be hard to prove formally.
  (Equivalence of all definitions of the binomial distribution.)

Submitted Dec 1st, 2025

Formalization of Brownian motion in Lean

Rémy Degenne, David Ledvinka, Etienne Marion, Peter Pfaffelhuber

Abstract. Brownian motion is a building block in modern probability theory. In this paper, we describe a formalization of Brownian motion using the LEAN theorem prover. We build on the existing measure-theoretic foundations in Lean's mathematical library, Mathlib, and we develop several key components needed for the construction of Brownian motion, including the Carathéodory and Kolmogorov extension theorems, Gaussian measures in Banach spaces, and the Kolmogorov-Chentsov theorem for path continuity.

Optimal strategies in the all-heads coin game

Verified in Lean 4, written with Claude Opus 4.7

Peter Pfaffelhuber

University of Freiburg

April 30, 2026

The game

- n coins; each flip: (H) with prob. p, (T) with prob. $q = 1 - p$
- each round: flip the in-play coins; keep ≥ 1 head (H)
- kept coins move into the right column and carry over
- bottom row all green $\Rightarrow$ *win*; no head $\Rightarrow$ *lose*

Strategy One

R1 (H)(T)(H)(T)

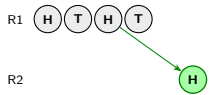
Strategy All

R1 (H)(T)(H)(T)

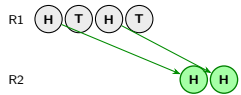
The game

- n coins; each flip: H with prob. p, T with prob. $q = 1 - p$
- each round: flip the in-play coins; keep ≥ 1 head H
- kept coins move into the right column and carry over
- bottom row all green $\Rightarrow$ *win*; no head $\Rightarrow$ *lose*

Strategy One



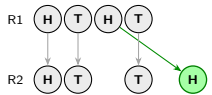
Strategy All



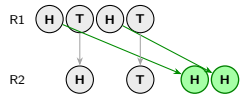
The game

- n coins; each flip: H with prob. p, T with prob. $q = 1 - p$
- each round: flip the in-play coins; keep ≥ 1 head H
- kept coins move into the right column and carry over
- bottom row all green $\Rightarrow$ *win*; no head $\Rightarrow$ *lose*

Strategy One



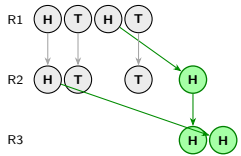
Strategy All



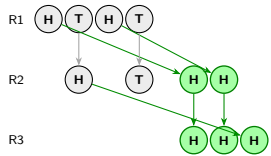
The game

- n coins; each flip: H with prob. p, T with prob. $q = 1 - p$
- each round: flip the in-play coins; keep ≥ 1 head H
- kept coins move into the right column and carry over
- bottom row all green $\Rightarrow$ *win*; no head $\Rightarrow$ *lose*

Strategy One



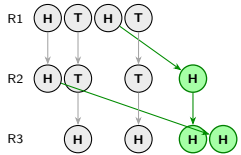
Strategy All



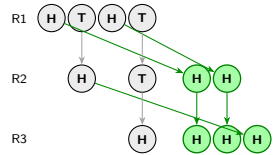
The game

- n coins; each flip: H with prob. p, T with prob. $q = 1 - p$
- each round: flip the in-play coins; keep ≥ 1 head H
- kept coins move into the right column and carry over
- bottom row all green $\Rightarrow$ *win*; no head $\Rightarrow$ *lose*

Strategy One



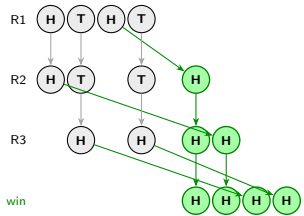
Strategy All



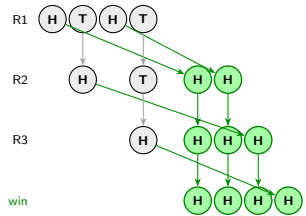
The game

- n coins; each flip: H with prob. p, T with prob. $q = 1 - p$
- each round: flip the in-play coins; keep ≥ 1 head H
- kept coins move into the right column and carry over
- bottom row all green $\Rightarrow$ *win*; no head $\Rightarrow$ *lose*

Strategy One



Strategy All



The Bellman equation

$w_{n,p}$ = optimal winning probability with n coins, $w_{0,p} := 1$.

$$w_{n,p} = p^n + \sum_{j=1}^{n-1} \binom{n}{j} p^{n-j} q^j \max_{j \le m \le n-1} w_{m,p}$$

- p^n: all heads $\Rightarrow$ immediate win
- j tails: choose remaining $m \in \{j, \ldots, n-1\}$
- player maximises $w_{m,p}$

The fair case $p = \frac{1}{2}$

Theorem

$w_{n,1/2} = \frac{1}{2}$ for every $n \geq 1$.

More: every policy taking the immediate win at $k = n$ has $w_{n,1/2} = \frac{1}{2}$.

Proof. Strong induction on n; key identity $\sum_{j=1}^{n-1} \binom{n}{j} 2^{-n} = 1 - 2^{1-n}$:

$$w_{n,1/2} \;=\; 2^{-n} + \tfrac{1}{2}\bigl(1 - 2^{1-n}\bigr) \;=\; \tfrac{1}{2}. \qquad \square$$

- symmetry axis
- all reasonable strategies optimal

Above: $p > \frac{1}{2}$

Theorem

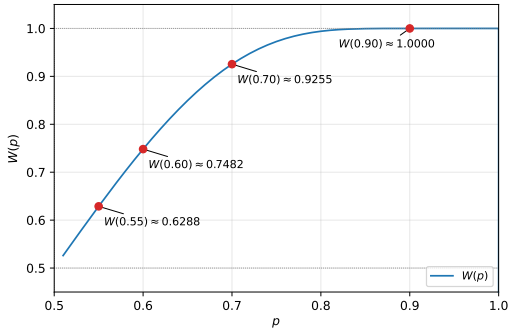
For $p > \frac{1}{2}$, $n \geq 2$:

(i) $w_{n,p} > w_{n-1,p}$

(ii) ONE optimal: $w_{n,p} = a_{n,p}$

$\Rightarrow W(p) := \lim_n w_{n,p}$ exists,
$p \leq W(p) < 1$,

$$W(p) = \sum_{k \geq 1} p^k \prod_{j>k} (1 - p^j - q^j).$$



Proof idea: joint induction with $w_{n-1,p} < \dfrac{p^n}{p^n + q^n}$.

Workflow: human + Claude Opus 4.7

Manuscript, code, Lean formalisation: produced *interactively*.

Author

- question: $w_{n,p}$ vs. $v_{n,p}$
- perturbation in $\delta = \frac{1}{2} - p$
- joint induction
- decision to formalise
- review every step

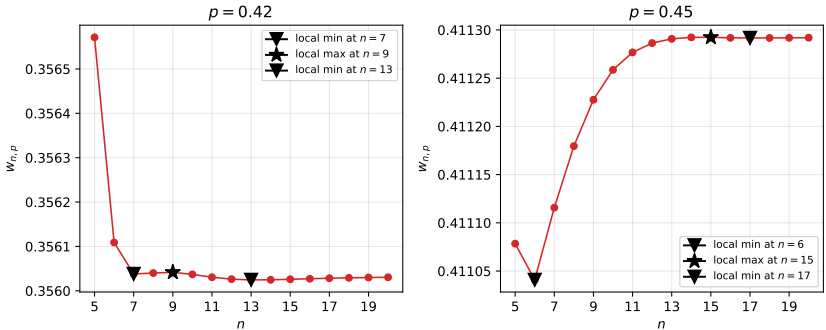
Claude

- drafts prose
- proposes / debugs Lean tactics
- picks Mathlib lemmas
- spots cubic / Tannery
- numerics, plots

Loop: pick result $\to$ draft $\to$ lake build $\to$ review $\to$ commit.

Log: journal.md $\cdot$ ≈ 3700 lines Lean $\cdot$ 0 sorry $\cdot$ 0 custom axioms.

Below: $p < \frac{1}{2}$ is hard



- local minima & maxima drift with p
- neither ONE nor ALL optimal in general
- our route: perturbation around $p = \frac{1}{2}$, $p = \frac{1}{2} - \delta$

First-order asymptotics: c_n

Deficit $\Delta_{n,p} := \frac{1}{2} - w_{n,p}$, $p = \frac{1}{2} - \delta$:

$$\Delta_{n,p} \;=\; c_n\,\delta + O(\delta^2), \quad c_1 = 1,$$

$$\boxed{c_n \;=\; \frac{n}{2^{n-1}} + \frac{1}{2^n}\sum_{j=1}^{n-1}\binom{n}{j}\min_{j\le m\le n-1} c_m} \quad (n \ge 2).$$

$c_1 = 1,\;\; c_2 = \frac{3}{2}$

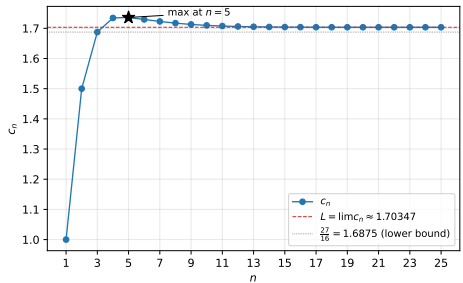
$c_3 = \frac{27}{16}$

$c_4 = \frac{111}{64}$

$c_5 = \frac{3555}{2048}$ (max)

$c_6 = \frac{113337}{65536}$

$c_n \to L \approx 1.7035$



Shape of $w_{n,p}$ at first order

Corollary ($p = \frac{1}{2} - \delta$, $\delta > 0$ small)

(i) $w_{n-1,p} - w_{n,p} = (c_n - c_{n-1})\,\delta + O(\delta^2)$

(ii) strict local *minimum* at $n = 5$

(iii) no local maximum at first order

Proof idea. Joint induction on n over four statements:

(a) *collapse:* $\min_{j \le m \le n-1} c_m = \begin{cases} c_j & j \le 3 \\ c_{n-1} & j \ge 4 \end{cases} \quad (n \ge 7)$

(b) *lower bound:* $c_n \ge \frac{27}{16} \quad (n \ge 4)$

(c) *strict decrease:* $c_n < c_{n-1} \quad (n \ge 6)$

(d) *linear recursion:* $c_n = A_n + (1 - B_n)\,c_{n-1} \quad (n \ge 7)$

Lean 4: Challenge.lean

Trust surface = Challenge.lean + Defs.lean.
Comparator: only propext, Quot.sound, Classical.choice.

```
-- Defs.lean : Bellman definition of w
noncomputable def w (p : ℝ) : ℕ → ℝ
 | 0     => 1
 | n + 1 =>
   p ^ (n + 1)
     + ∑ j ∈ (Ico 1 (n + 1)).attach,
       (Nat.choose (n + 1) j.val : ℝ)
         * p ^ (n + 1 - j.val) * (1 - p) ^ j.val
         * (Ico j.val (n + 1)).attach.sup' _ (fun m => w p m.val)

-- Challenge.lean : one of 16 headline statements
theorem w_local_min_at_five :
 ∃ δ₀ : ℝ, 0 < δ₀ ∧ ∀ δ, 0 < δ → δ < δ₀ →
   w (1/2 - δ) 5 < w (1/2 - δ) 4 ∧
   w (1/2 - δ) 5 < w (1/2 - δ) 6
```

16 theorems · 0 sorry · 0 custom axioms · github.com/pfaffelh/coins